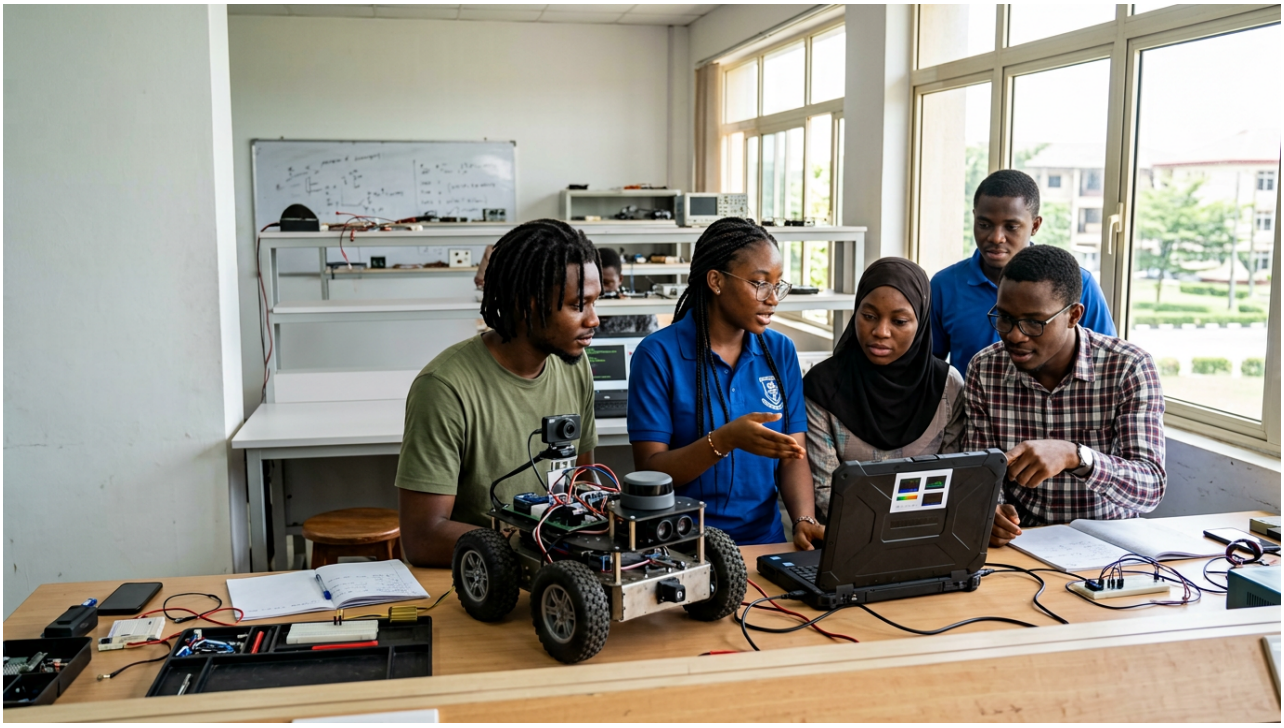
# AI.101 — Introduction to Artificial Intelligence



*Illustrative engineering-laboratory image; not documentary evidence of a specific university or event.*

## Narration manuscript · Module 2: Task Environments and Problem Formulation

**Academic level:** Bachelor of Artificial Intelligence and Robotics Engineering
**Content policy:** No faith-related content.

## 1. From an interesting idea to an engineering task

In our first module, we introduced an agent as a system that receives percepts from an environment and chooses actions. We also saw that a vague goal such as “make the robot intelligent” is not yet an engineering specification. Today we will make the task precise enough that different designers can reason about the same problem, compare alternatives, and evaluate whether a proposed system works.

A task environment is the setting in which an agent operates, including the conditions that affect what it observes, what it can do, and what counts as success. It includes more than the physical room or map. It may include other agents, communication delays, safety rules, available power, user expectations, and the system’s operational limits. A good description explains both what is inside the system boundary and what remains outside it.

This boundary is a design choice, not a fact handed to us automatically. Consider a wheeled robot that delivers a package in a university building. If we draw the boundary around the robot alone, we may describe cameras, motors and onboard software but ignore the building map, elevator service, people who open doors, and the technician who responds to faults. If we expand the boundary, we recognize those dependencies and can ask what happens when a network connection fails or an elevator is unavailable. The wider boundary can make a problem look more complex, but it also makes hidden assumptions visible.

A task specification is not an algorithm. It tells us what problem we mean to solve. A solution method is a procedure for solving it. An implementation is the actual software and hardware that runs. Keeping these levels separate is essential. If a task has not been specified, an algorithm cannot be judged fairly. If an algorithm is selected too early, it can distort the requirements to fit the tool the designer happens to know.

## 2. PEAS: a compact task description

A widely used way to describe an agent task is PEAS. The letters stand for **Performance measure, Environment, Actuators, and Sensors**. The framework is deliberately simple. It prompts us to discuss outcomes, operating conditions, available actions and available information before we decide how the agent should reason.

The **performance measure** defines what we value in the agent’s behavior. A useful measure often contains several components. For a delivery robot, we might care about successful delivery, safe operation, travel time, energy consumption, package condition, accessibility and the need for human intervention. These components can conflict. A route that is short may be steep; a route that is quick may pass through a crowded space; a route that saves energy may leave too little reserve for a detour.

The **environment** describes the world in which the agent must perform. Include relevant objects, people, rules and conditions. For a campus delivery robot, this may include corridors, doors, ramps, elevators, pedestrians, temporary obstacles, floor surfaces, lighting, building opening hours and wireless coverage. The environment description should not assume that everything remains in its expected state. Doors can be closed, corridors can be blocked, and maps can be incomplete.

The **actuators** are the means by which an agent affects the environment. In a mobile robot, these may include wheel motors, steering, a brake, a display, a speaker, a status light or a network request to an elevator controller. A software agent may have different actuators: sending a message, updating a database, changing a setting or requesting human approval. State the action space clearly. If an agent cannot open doors, it should not be planned as though door opening were available.

The **sensors** are the channels through which the agent receives information. A robot might use a camera, lidar, ultrasonic range sensors, wheel encoders, an inertial measurement unit, bump switches and battery telemetry. A service agent might receive text, timestamps, user selections, database records and error codes. A sensor provides measurements, not perfect access to reality. Cameras can be affected by glare; encoders can drift when wheels slip; a database record can be out of date.

Reveal four PEAS panels. For each panel, put the campus delivery robot example on one side and the design question on the other: What counts as success? What can change? What actions exist? What can be sensed?

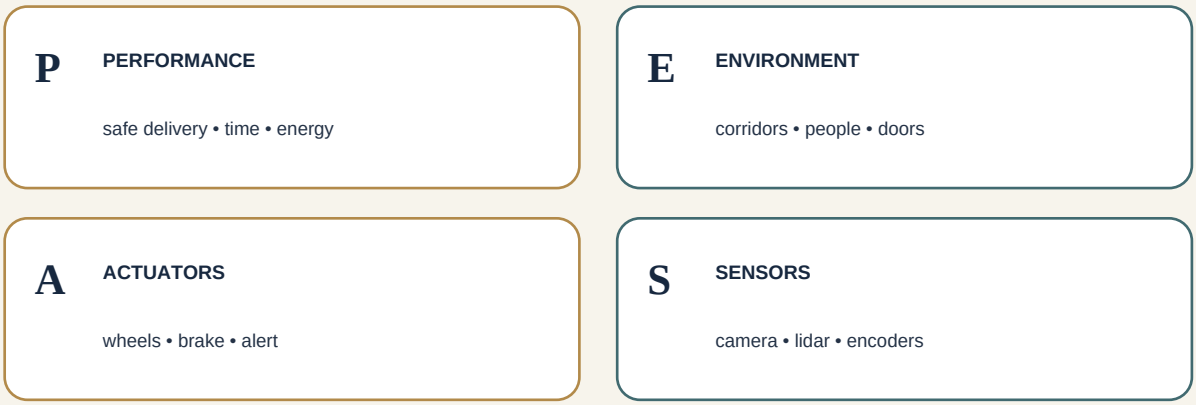



*Diagram: PEAS task specification*

The value of PEAS is not that every box has a single correct answer. Its value is that it exposes omissions. If the performance measure includes speed but not collision risk, we should question the measure. If the environment includes pedestrians but the sensor list has no way to detect them, we have a system gap. If the action list contains “ask for help” but the design has no operator channel, the specification and implementation disagree.

## 3. Worked specification: campus delivery

Let us work through a task together. A robot must carry a sealed package from a robotics laboratory to a staffed reception desk. It should arrive during the building’s public hours, preserve the package, and report completion to an operator.

Start with the performance measure. We can separate hard constraints from soft preferences. Hard constraints might include: no contact with people; package remains secured; destination is correct; the robot obeys an approved speed limit; and the system stops safely when it cannot proceed. Soft preferences might include minimizing travel time, energy use, noise and unnecessary operator requests. This distinction is useful because it prevents a scoring formula from accidentally allowing a small improvement in speed to compensate for a serious safety violation.

Next, describe the environment. The building contains corridors, rooms, access-controlled doors, ramps, elevators, people, other service robots, furniture and changing signs. The floor plan may be known, but the exact locations of people and temporary obstacles are not. The system operates in daylight and artificial lighting, and it may encounter reflective floor surfaces. The wireless network is usually available but cannot be assumed to be perfect. The building may close while the robot is still on a task.

Now list actuators. The robot can command left and right wheel speeds, brake, sound a short alert, display a status message, request an elevator, and send an assistance request to an operator. It cannot physically move a heavy object blocking a corridor. It cannot compel a person to open a door. Those limitations belong in the task model because they affect which plans are feasible.

Finally, list sensors. A camera can identify landmarks and obstacles, lidar can estimate distances to nearby surfaces, wheel encoders can estimate wheel rotation, and an inertial sensor can measure changes in motion. A battery monitor reports available charge. A map provides expected corridor connectivity. A door-status service may provide information if it is online. The operator can supply a response, but that response may arrive after a delay.

We have not yet selected a navigation algorithm. That is intentional. We have described what the robot must do and what resources it has. We can now ask whether a route planner should optimize a weighted cost, whether certain restrictions should be hard rules, and how the system should behave when observations conflict.

Suppose the map says a corridor is open but lidar detects a barrier. The robot should not blindly trust the map. Suppose the camera sees a person but the image is blurred. The robot should not treat a low-confidence estimate as certainty. Suppose the elevator service fails. The planner should seek an accessible alternate route or ask for help rather than continuing to issue the same failed request. These are policy questions that arise from the task specification.

## 4. Classifying environments to anticipate difficulty

Task environments can be classified along several dimensions. These categories help us anticipate which methods may be appropriate, but they do not replace detailed analysis.

An environment is **fully observable** if the agent can access all information needed to determine the relevant state at each decision. It is **partially observable** if important aspects are hidden or uncertain. A board game with all pieces visible is close to fully observable. A robot navigating a busy hallway is partially observable: the robot cannot see around every corner, sensor readings are noisy, and the future movement of people is unknown.

An environment is **deterministic** when the next state is fully determined by the current state and chosen action. It is **stochastic** when outcomes involve uncertainty. A robot commanding a small wheel rotation may not move exactly as predicted because of wheel slip, uneven flooring or contact with an obstacle. Many practical environments are best treated as stochastic, even if the physical processes themselves follow laws, because the agent’s model and measurements are incomplete.

An environment is **episodic** when each decision can be evaluated independently of later decisions. In a sequence of image classifications, one classification may not affect the next. It is **sequential** when current actions affect future states and opportunities. Robot navigation is sequential: taking a detour changes travel time, energy reserve and future route choices. A poor action now can make later decisions harder.

An environment is **static** when it does not change while the agent is deciding. It is **dynamic** when it can change during deliberation or execution. A printed maze is static. A corridor with moving pedestrians is dynamic. In dynamic settings, the agent must consider the age of its observations. A correct camera reading from several seconds ago may no longer describe the current path.

An environment is **discrete** when relevant states and actions can be represented as distinct items, such as choosing among named rooms or selecting a direction from north, south, east and west. It is **continuous** when quantities vary over ranges, such as wheel speed, steering angle, time or distance. Many robots combine both: a route planner chooses a sequence of corridor segments while a controller adjusts continuous motor speeds.

Finally, an environment may involve a **single agent** or **multiple agents**. People, vehicles, software services and other robots can all affect outcomes. In multi-agent environments, actions by others may be cooperative, independent or adversarial. A mobile robot crossing a shared hallway must predict and respond to human movement without assuming that people will follow the robot’s preferred route.

A six-axis environment profile chart: observable/partially observable, deterministic/stochastic, episodic/sequential, static/dynamic, discrete/continuous, single/multi-agent. Place the campus robot along each axis and narrate why the task lies where it does.

The campus robot is partially observable, uncertain, sequential, dynamic, partly continuous and multi-agent. This profile tells us that a simple fixed sequence of motor commands is unlikely to be sufficient. The robot will need feedback, state estimation, planning, and safe responses to unexpected events. We have not yet proven which specific algorithms are best; we have identified properties that any reasonable design must address.

## 5. Performance measures and trade-offs

Performance measures deserve special attention because they shape behavior. If we reward only the number of completed deliveries, a system may rush, take unsafe shortcuts, or avoid asking for help even when assistance is necessary. If we reward only cautious stopping, the robot may never finish a task. If we reward only user satisfaction, the system may make promises it cannot keep. Measurement is a design decision with consequences.

One approach is to separate safety constraints from optimization goals. For example, define collision avoidance and legal speed as constraints that must never be traded away for a faster arrival. Then optimize travel time and energy within those bounds. Another approach is to use a weighted score, but weights must be justified and tested. A score can conceal a bad trade-off: if saving ten seconds is worth enough points, it could numerically compensate for an unacceptable risk. The designer must determine which quantities can be traded and which cannot.

Measurement should also include the denominator and operating conditions. “Ninety-nine successful runs” is incomplete unless we know how many total runs occurred, what environments were tested, how success was defined, and which cases were excluded. A system should be tested across variation in lighting, traffic, floor type, network availability and task distance. We will study evaluation more deeply later in the course, but the habit begins now: make the criteria explicit before claiming success.

## 6. Practical design exercise: PEAS worksheet

Choose one of these tasks: a line-following robot, an indoor delivery robot, an automated greenhouse monitor, or a document-triage assistant. Complete a PEAS table with at least four entries in each category. Then mark each item as known, uncertain or dependent on an external service.

For a line-following robot, the performance measure might include completing a track, staying within a safe speed, avoiding collisions and using limited battery power. The environment includes the line, floor texture, lighting, bends, intersections and unexpected obstacles. Actuators include left and right motors, a stop command and perhaps a status indicator. Sensors may include reflectance sensors, wheel encoders and a distance sensor. Notice how the task changes if the track can be wet or if the line can disappear at intersections.

After completing the table, classify the environment along the six dimensions. Explain at least one design implication for each classification. If the environment is partially observable, what state information is hidden? If it is dynamic, how quickly can it change? If it contains several agents, what assumptions are safe to make about their behavior?

Now write a brief failure response. What should the agent do when a sensor is unavailable, the task becomes impossible, or a safety constraint is approached? “Try again” is not always adequate. A good agent may stop, move to a safe state, report the problem, preserve evidence, and request human assistance.

## 7. Common mistakes and troubleshooting

A frequent error is to list technologies instead of task requirements. “Use lidar and deep learning” tells us what a designer wants to use, not what the robot must accomplish. State the task independently, then justify the technology.

Another error is to define the environment too narrowly. A robot that depends on a remote service must account for network delay and outages. An AI tool used by people must account for user interpretation and workflow. Boundaries should be explicit so that responsibilities do not disappear between components.

A third error is to make every objective a single weighted number. Some priorities, especially safety and legal compliance, should be represented as constraints or separate gates. If a single score is used, demonstrate that

the weights produce acceptable behavior in realistic scenarios.

A fourth error is to treat sensor measurements as ground truth. Sensors have noise, bias, latency, limited range and failure modes. Even a perfectly functioning sensor may not measure the quantity the designer actually needs. Document the measurement process and uncertainty.

Finally, designers sometimes specify a perfect environment: clear corridors, reliable maps, cooperative pedestrians, constant lighting and uninterrupted networking. This produces a demonstration that works only in the demonstration. A stronger specification lists ordinary variability and tests the system against it.

## 8. Knowledge check and assignment

Answer these questions before continuing:

1. What does PEAS stand for, and what kinds of design questions does it prompt?
2. How can the chosen system boundary hide external dependencies?
3. Why should a safety requirement not always be represented as a weighted preference?
4. Give an example of a partially observable and dynamic environment.
5. How does sequential structure change the way an agent’s actions should be evaluated?
6. Why is a list of sensors and algorithms not a complete task specification?

For the assignment, prepare a two-page task-environment specification for a small autonomous robot or AI software agent. Include a PEAS table, a boundary diagram, an environment classification profile, hard constraints, soft objectives, at least five failure conditions, and a preliminary test plan. State assumptions separately from verified facts. Do not select a complex learning model unless you can explain why simpler approaches are inadequate.

## Module summary

Today we moved from a broad AI idea to an engineering task specification. PEAS helps us record performance measures, environment, actuators and sensors. Environment classification helps us reason about observability, uncertainty, sequential effects, changing conditions, continuous variables and multiple agents. We also separated task, method and implementation; examined a campus delivery robot; and saw why safety constraints, sensor limitations and external dependencies must be written down.

In the next module, we will formalize problem solving. We will represent states and actions, define initial and goal states, and introduce state-space search. The task specification from today will guide those choices: search is only meaningful when we know what the states represent, which actions are possible, and what outcome counts as a solution.